White box test case generation tool

Team ID:10 Team Members：冯俊财2353924
谢雨霏2354264、纪鹏2351869、金子涵2351442、于伊莲2352993

2026.4.20

1.Project Background & Tool Design

➤ 1.1 Manual White-box Testing

Methodology (3 Steps):

- Construct Control Flow Graph (CFG)
- Compute Cyclomatic Complexity
- Derive independent paths and design test cases

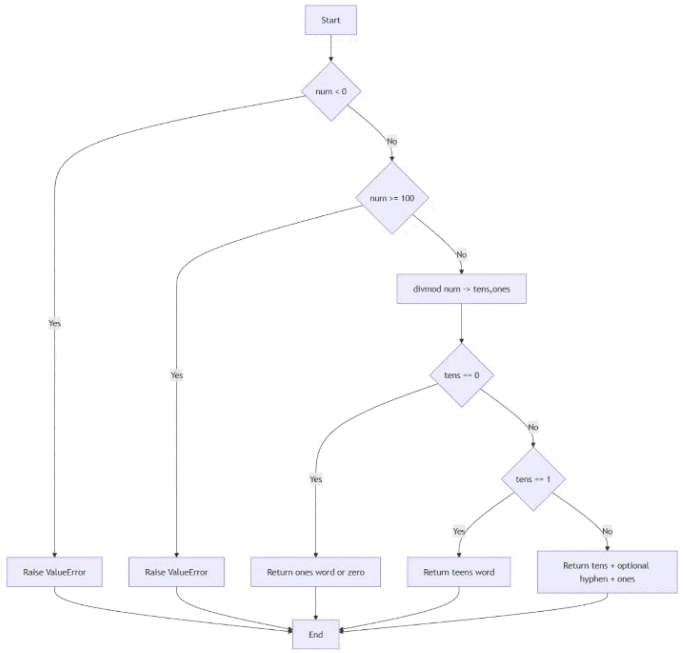
Test Subjects & Complexity(V(G)):

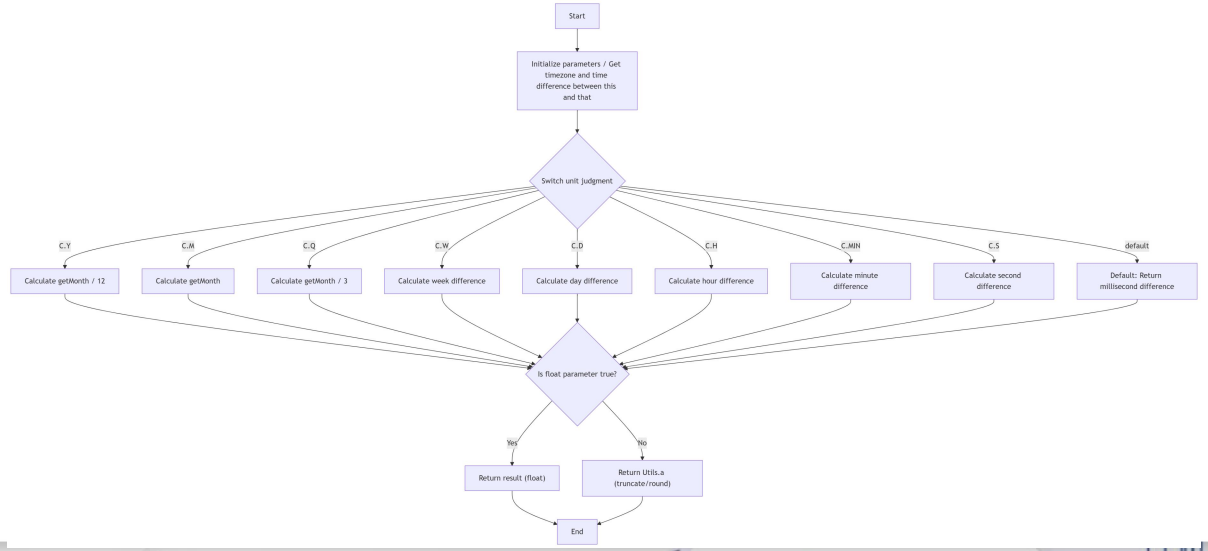
Python (convert_number_to_words.py)

- max_value (4)
- convert_small_number (5)
- convert_number (6)

JavaScript (dayjs)

- parseDate (7)
- parseLocale (7)
- Diff (10)

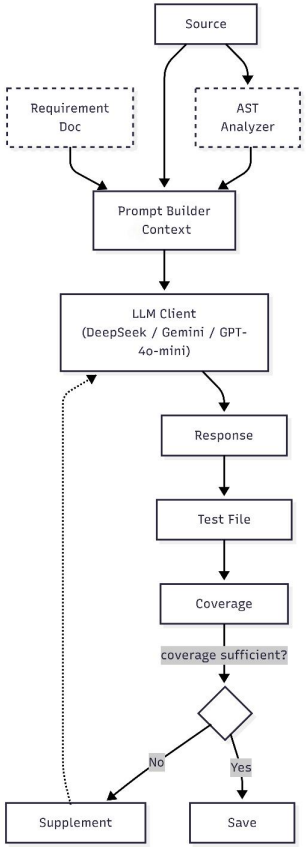




1.Project Background & Tool Design

1.2 AI-based Test Generation Tool

Tool Architecture



Prompt Modes:

- General generation for standard targets
- AST-assisted mode for Python analysis
- Script generation for complex cases
- Supplement prompts for closed-loop refinement

2. Experimental Results & Analysis

Evaluation Metrics:

- Coverage (statement / branch)
- Accuracy (pass rate)
- Test code quality (syntax, logic, framework compatibility)

Main Findings

- Closed-loop refinement improves coverage by generating tests for uncovered code
- AST guidance improves test correctness by providing structured branch information
- Direct script generation improves JavaScript results on complex cases better than JSON-based conversion

Representative Result

Aspect	Python Project	JavaScript Project
Coverage	85.9%–91.0% (stable)	0%–97.1% (mostly <50%)
Accuracy	53.33%–95.83% (mostly >80%)	0%–100% (mostly <10%)
Errors	Incorrect method invocation	Syntax & framework misuse

3. Challenges & Engineering Solutions

➤ 3.1 JSON Limitation → Direct Script Generation

Problem:

- JSON cannot represent chained calls (e.g., dayjs().diff())

Solution:

- Introduced script mode
- LLM directly generates executable test scripts
- Closed-loop adds only new fragments

➤ 3.2 Missing Syntax in AST → Analyzer Extension

Problem:

- Our initial AST analyzer missed Python 3.10 match-case branches

Solution:

- Extended analyzer to recognize match-case branches

3. Challenges & Engineering Solutions

➤ 3.3 Context Compression → Invalid Test Cases

Problem:

- Partial code leads to invalid inputs (e.g., convert_small_number(100) blocked by validation)

Solution:

- Provide richer function context
- Make input constraints more explicit in prompts

Partial code

```
# 只给函数体，没有入口校验信息
def convert_small_number(num: int) -> str:
    tens = num // 10
    ones = num % 10
    # ... 转换逻辑
```

full function context

```
def convert_small_number(num: int) -> str:
    """转换小于100的数字为英文单词"""
    # 入口校验
    if num < 0:
        raise ValueError("只接受非负整数")
    if num >= 100:
        raise ValueError("只转换小于100的数字")

    # 转换逻辑
    tens = num // 10
    ones = num % 10
    # ... 完整实现
```

3. Challenges & Engineering Solutions

➤ 3.4 LLM “Common Sense” vs Code Reality

Problem:

- LLM assumes rounding behavior
- Actual implementation follows library-specific flooring logic

Solution:

- Describe library behavior more explicitly
- Use toBeCloseTo() for floating-point outputs when appropriate

False assertion

```
// LLM认为diff会四舍五入
expect(base.diff('2024-04-15',
'month', true)).toBe(1.5)
```

Correct assertion

```
// Dayjs实际使用Math.floor
expect(base.diff('2024-04-15',
'month', true)).toBeCloseTo(1.548,
3)
```

3. Challenges & Engineering Solutions

➤ 3.5 Output Instability → Fault-tolerant Engineering

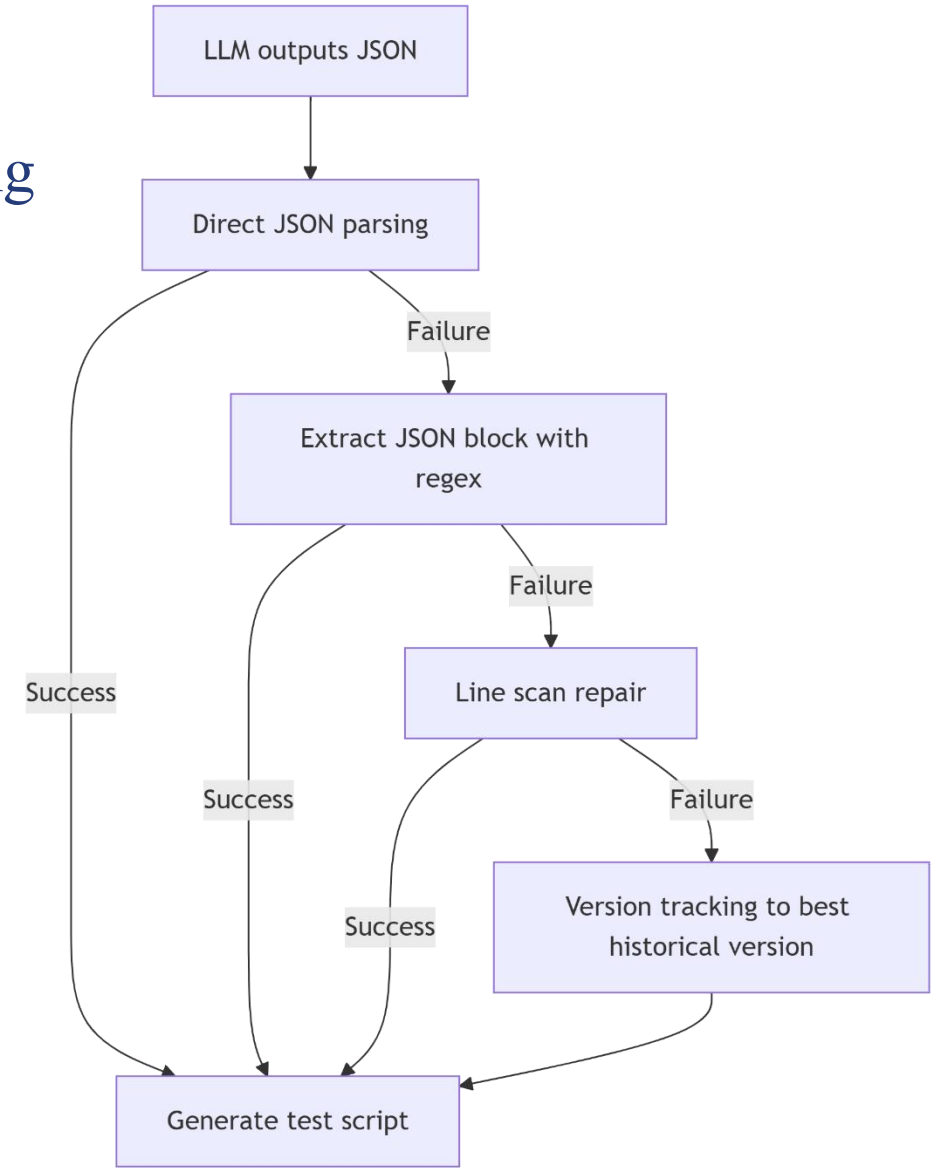
Problem:

- Extra text breaks JSON parsing
- Syntax errors break entire test suite

Solution:

- Multi-layer parsing:
  direct load → regex → line repair
- Version tracking & rollback

Analysis and Rollback Flowchart



4. Conclusion & Future Work

4.1 Key Conclusions

- AI-generated testing is comparable to manual testing for structured Python code (>90% coverage)
- Still immature for complex JavaScript (e.g., chaining, prototype inheritance)
- Key insight: Robust engineering systems matter more than perfect single LLM outputs

4.2 Practical Contributions

- Built a complete LLM-based white-box testing tool
- Achieved full pipeline automation: Code analysis → test generation → closed-loop refinement
- Key innovations: AST-integrated analysis, Adaptive output modes, Multi-LLM comparison, Robust rollback mechanism

4.3 Future Development Directions

- Enhance AST analysis to support more syntax and improve structured path coverage
- Combine LLM with symbolic execution and fuzzing for better edge-case coverage
- Optimize prompt design (few-shot & CoT) to improve accuracy and reduce hallucination

Thank you for listening

Team ID:10 Team Members：冯俊财2353924
谢雨霏2354264、纪鹏2351869、金子涵2351442、于伊莲2352993

2026.4.20